# Homework 3 Part A: Learning on Graphs by Hand

A first mathematical introduction to graph neural networks

**Theme:** We will learn graph neural networks by *computing on small graphs by hand.*
**Inspirationr:** This homework is inspired by the survey *Theory of Graph Neural Networks: Representation and Learning* by Stefanie Jegelka, especially its discussion of message passing, permutation invariance, Weisfeiler–Leman, spectral GNNs, expressive power, and generalization.
**What you may use:** class notes, definitions in this handout, and basic linear algebra.
**What you may not assume:** prior knowledge of abstract algebra, measure theory, or advanced machine learning theory.
**What to submit:** a .tex file converted into a pdf **Score:** [150pts (Part A) + 150 (Part B)] /20

## Why this homework exists

Graph neural networks are often presented as black-box machine learning models. In reality, many of their core ideas are concrete and mathematical:

- a graph can be encoded by matrices,
- a node can repeatedly aggregate information from its neighbors,
- this repeated aggregation has limits,
- those limits are closely related to the Weisfeiler–Leman graph isomorphism test,
- and some graph neural networks can also be understood through spectral graph theory.

This part of the homework is designed to make those ideas concrete. If your time allows it, I encourage you to try to implement a GNN in pytorch geometric.

## Notation and mini-reference

A graph is denoted by $G = (V, E)$. We write $V = \{1, \dots, n\}$ for the set of nodes and $E$ for the set of edges. The **adjacency matrix** $A \in \mathbb{R}^{n\times n}$ is defined by

$$A_{ij} = \begin{cases} 1 & \text{if } (i,j) \in E, \\ 0 & \text{otherwise.} \end{cases}$$

If each node has a scalar feature $x_i$, then the node features can be collected into a vector

$$\mathbf{x} = (x_1, \dots, x_n)^\top \in \mathbb{R}^n.$$

More generally, if each node has a $d$-dimensional feature vector, then the features form a matrix $X \in \mathbb{R}^{n\times d}$.

A basic message passing GNN layer has the form

$$m_v^{(t)} = \sum_{u\in\mathcal{N}(v)} h_u^{(t-1)}, \qquad h_v^{(t)} = \sigma\Big(W_1 h_v^{(t-1)} + W_2 m_v^{(t)}\Big),$$

where $h_v^{(t)}$ is the representation of node $v$ after $t$ rounds, $\mathcal{N}(v)$ is the neighborhood of $v$, and $\sigma$ is a nonlinearity such as ReLU. In this homework, we will mostly use *very simple toy versions* of this update so that you can compute them by hand.

A graph-level prediction must be **permutation invariant**: relabeling the nodes should not change the answer.

## A note on style

For many questions below, the goal is not only to get the final answer, but also to explain *why* the answer is reasonable. Show your intermediate steps.

## Part I. Graphs, matrices, and neighborhood aggregation

### Problem 1. Reading a graph as a matrix (10 points)

Consider the path graph on four nodes

$$1 - 2 - 3 - 4.$$

(a) Write its adjacency matrix $A$.

(b) Let the initial node feature vector be

$$\mathbf{x} = (1, 0, 2, 1)^\top.$$

Compute $A\mathbf{x}$.

(c) Explain in words what the $i$th entry of $A\mathbf{x}$ means.

(d) Compute $A^2\mathbf{x}$ and interpret it informally.

**Hint.** Multiplying by $A$ mixes information across one edge. Multiplying by $A^2$ mixes information across walks of length 2.

### Problem 2. One round of message passing by hand (10 points)

On the same graph, let the initial scalar node features be

$$h^{(0)} = (1, 0, 2, 1)^\top.$$

Define a toy GNN update by

$$m_v^{(1)} = \sum_{u \in \mathcal{N}(v)} h_u^{(0)}, \qquad h_v^{(1)} = h_v^{(0)} + m_v^{(1)}.$$

(a) Compute $m_v^{(1)}$ for each node $v$.

(b) Compute $h_v^{(1)}$ for each node $v$.

(c) Write the vector $h^{(1)}$.

(d) Explain why this update only uses information from the 1-hop neighborhood.

### Problem 3. Two rounds of message passing by hand (10 points)

Continue Problem 2 for one more round:

$$m_v^{(2)} = \sum_{u \in \mathcal{N}(v)} h_u^{(1)}, \qquad h_v^{(2)} = h_v^{(1)} + m_v^{(2)}.$$

(a) Compute $m_v^{(2)}$ and $h_v^{(2)}$ for all nodes.

(b) Which nodes can influence node 2 after two rounds?

(c) Explain in words why $t$ rounds of message passing can only depend on information inside the $t$-hop neighborhood.

### Problem 4. ReLU version of a manual GNN (10 points)

Consider the graph

$$1 - 2 - 3$$

and initialize

$$h^{(0)} = (-1, 2, -2)^\top.$$

Now define

$$m_v^{(1)} = \sum_{u \in \mathcal{N}(v)} h_u^{(0)}, \qquad h_v^{(1)} = \mathrm{ReLU}\big(h_v^{(0)} + m_v^{(1)}\big),$$

where $\mathrm{ReLU}(z) = \max\{z, 0\}$.

(a) Compute $m^{(1)}$.

(b) Compute $h^{(1)}$.

(c) Briefly explain how the nonlinearity changes the calculation compared with Problems 2–3.

---

## Part II. Weisfeiler–Leman color refinement by hand

### Background: the 1-WL algorithm

The **1-dimensional Weisfeiler–Leman algorithm** (often called **1-WL** or **color refinement**) is an iterative graph procedure.

**Input:** a graph whose nodes each start with a label or color. If no labels are given, start by giving every node the same color.

**Update rule:** at step $t$, each node replaces its current color by a new color determined by:

$$\big(\text{current color of the node},\ \text{multiset of colors of its neighbors}\big).$$

Then all distinct such pairs are renamed by fresh colors.

**Stopping rule:** stop when the multiset of colors no longer changes.

**Important idea:** 1-WL does not remember exact node identities. It only refines nodes by local structural patterns.

## Problem 5. Run 1-WL on a path graph (10 points)

Consider the path graph on 4 nodes:

$$1 - 2 - 3 - 4.$$

Initially, all nodes have color $a$.

(a) Write the color of each node at iteration $t = 0$.

(b) Run one round of 1-WL. For each node, write the pair

$$(\text{current color}, \text{multiset of neighbor colors}).$$

Then assign new names to the resulting colors.

(c) Run a second round.

(d) Has the coloring stabilized? Explain.

(e) Which nodes end up with the same color? Why does that make sense geometrically?

## Problem 6. Run 1-WL on a cycle graph (10 points)

Consider the cycle graph $C_4$ on four nodes:

$$1 - 2 - 3 - 4 - 1.$$

Initially, all nodes have color $a$.

(a) Run two rounds of 1-WL.

(b) What happens to the colors?

(c) Explain why 1-WL cannot distinguish the nodes in this graph.

## Problem 7. Compare two graphs using 1-WL (10 points)

Let

- $G_1$ be the path graph on four nodes, and
- $G_2$ be the cycle graph on four nodes.

Start with all nodes having the same color.

(a) Run 1-WL on both graphs for at least two rounds.

(b) At each round, write down the *multiset of node colors* for each graph.

(c) At what point can 1-WL tell these graphs apart?

(d) Explain why comparing multisets of colors is natural for graph isomorphism testing.

### Problem 8. A graph pair that 1-WL struggles with (10 points)

Consider the following two graphs on 6 nodes:

- $G_3$: the cycle $C_6$,
- $G_4$: the disjoint union of two triangles.

(a) Write the degree of every node in each graph.

(b) Start 1-WL with one identical color on all nodes and run two rounds.

(c) What color pattern do you get in each graph?

(d) Can 1-WL distinguish $G_3$ and $G_4$ in this setup?

(e) Why is this a warning sign for message passing GNNs?

**Hint.** In both graphs, every node has degree 2, but the global structure is different.

### Problem 9. Manual GNN versus 1-WL (10 points)

Return to the graphs $G_3$ and $G_4$ from Problem 8. Give every node initial feature 1, and use the toy GNN update

$$h_v^{(t)} = h_v^{(t-1)} + \sum_{u \in \mathcal{N}(v)} h_u^{(t-1)}.$$

(a) Compute $h_v^{(1)}$ for every node in both graphs.

(b) Compute $h_v^{(2)}$ for every node in both graphs.

(c) What do you observe?

(d) Explain, at an intuitive level, why message passing GNNs and 1-WL are closely related.

---

## Part III. Theory: what GNNs can and cannot represent

### Mini-theory box

A message passing GNN updates each node by combining its current representation with a permutation-invariant function of its neighbors. One standard form is

$$m_v^{(t)} = \phi_1\left(\sum_{u \in \mathcal{N}(v)} \phi_2(h_u^{(t-1)}, h_v^{(t-1)}, w(u,v))\right),$$

followed by an update such as

$$h_v^{(t)} = \sigma\Big(W_1 h_v^{(t-1)} + W_2 m_v^{(t)}\Big).$$

A graph-level output is then produced by a permutation-invariant readout, for example a sum or average over nodes.

One of the key theoretical insights is:

> If a message passing GNN can distinguish two graphs, then the 1-WL algorithm can also distinguish them.

So message passing GNNs are powerful, but not unlimited.

## Problem 10. Permutation invariance (10 points)

Suppose you relabel the nodes of a graph. The adjacency matrix changes from $A$ to $PAP^\top$, where $P$ is a permutation matrix.

(a) Explain in plain language what a node relabeling does.

(b) Why should a graph classification model give the same answer before and after relabeling?

(c) Give one example of a graph-level readout that is permutation invariant.

(d) Give one example of a readout that is *not* permutation invariant.

## Problem 11. Why sum aggregation matters (optional challenge)

A theorem discussed in the survey says that multiset functions can often be represented in the form

$$G(S) = \phi_1\left(\sum_{s\in S} \phi_2(s)\right).$$

(a) What is a multiset? How is it different from an ordinary set?

(b) Why are neighborhoods naturally multisets rather than ordered lists?

(c) Why is a sum more natural than concatenating neighbors in a graph setting?

(d) Explain why averaging can lose information that summing keeps. Give a small numerical example.

## Problem 12. Computation trees and locality (optional challenge)

A useful way to think about message passing is through a **computation tree**: after $t$ rounds, the representation of a node is built from repeated neighborhood aggregation up to depth $t$.

(a) Draw the depth-2 computation tree rooted at node 2 in the path graph $1 - 2 - 3 - 4$.

(b) Which original graph nodes appear in that tree?

(c) Explain why two nodes with the same local unfolding can be hard for a message passing GNN to distinguish.

(d) Why does this suggest that some global graph properties are difficult for message passing GNNs?

### Problem 13. Limits of message passing (optional challenge: bonus: 5pts)

The survey discusses several graph properties that message passing GNNs have trouble deciding exactly.

(a) In your own words, explain why a purely local aggregation scheme may struggle with a global property such as graph diameter.

(b) Why might counting certain cycles also be hard?

(c) Give one graph property that seems local and one that seems global.

(d) For each, explain whether you would expect a shallow GNN to do well or poorly.

### Problem 14. Higher-order ideas (optional challenge)

The survey explains that one way to strengthen message passing is to move from single nodes to *pairs* or *k-tuples* of nodes.

(a) Why might considering node pairs capture more structure than considering one node at a time?

(b) What is the main computational drawback of doing this?

(c) Explain this tradeoff in your own words.

---

## Part IV. Spectral graph theory for beginners

### Mini-theory box

The **graph Laplacian** of an undirected graph is

$$L = D - A,$$

where $D$ is the diagonal degree matrix and $A$ is the adjacency matrix.

A spectral GNN can be thought of as learning a function of the Laplacian, for example

$$F(G) = p(L)X,$$

where $p$ is a polynomial. This means that graph learning can also be viewed through eigenvalues and eigenvectors.

For this homework, you only need the following facts:

- $D$ is diagonal with $D_{ii} = \deg(i)$,
- $L = D - A$,
- if $L\mathbf{1} = 0$, then the constant vector is an eigenvector with eigenvalue 0,
- multiplying by $L$ compares a node's value to its neighbors' values.

### Problem 15. Build the Laplacian (10 points)

For the path graph $1 - 2 - 3 - 4$:

(a) Write the adjacency matrix $A$.

(b) Write the degree matrix $D$.

(c) Compute the Laplacian $L = D - A$.

(d) Verify directly that $L\mathbf{1} = 0$.

### Problem 16. Laplacian as smoothing difference operator (10 points)

Let

$$\mathbf{f} = (1, 3, 3, 1)^\top$$

on the path graph $1 - 2 - 3 - 4$.

(a) Compute $L\mathbf{f}$.

(b) Look at each coordinate of $L\mathbf{f}$ and explain what it says about that node relative to its neighbors.

(c) Which nodes look "flat" relative to neighbors and which look "different"?

### Problem 17. Compare adjacency mixing and Laplacian mixing (10 points)

Using the same graph and feature vector from Problem 16:

(a) Compute $A\mathbf{f}$.

(b) Compare $A\mathbf{f}$ with $L\mathbf{f}$.

(c) In words, how is multiplying by $A$ different from multiplying by $L$?

### Problem 18. A first spectral filter (10 points)

Define a simple spectral-style filter by

$$\mathbf{g} = (I - \alpha L)\mathbf{f}$$

with $\alpha = \frac{1}{2}$.

(a) Using your answer from Problem 16, compute $\mathbf{g}$.

(b) Does this operation make neighboring values more similar or less similar?

(c) Why might such an operation be useful in graph learning?

(d) Can you think of a connection with message passing in graph neural networks?

# Part V. Learning and generalization: conceptual only, stuff to think about, you won't get graded on this

### Mini-theory box

The survey also discusses learning theory for GNNs. You do *not* need to prove advanced theorems here. The point is to understand the questions.

When training a GNN, we minimize empirical risk,

$$\widehat{R}(F) = \frac{1}{N} \sum_{i=1}^{N} \ell\big(G_i, y_i, F(G_i)\big),$$

but what we really care about is population risk,

$$R(F) = \mathbb{E}_{(G,y)}\, \ell\big(G, y, F(G)\big).$$

The gap between them is called the **generalization gap**.

The survey explains that generalization can be studied through ideas such as:

- model complexity,
- Rademacher complexity,
- neural tangent kernel approximations,
- and **algorithmic alignment**: when the architecture mirrors the structure of the task.

### Problem 20. Population risk versus empirical risk

Answer in complete sentences.

(a) What is the difference between empirical risk and population risk?

(b) Why can a model do very well on training graphs but still fail on new graphs?

(c) Why is this issue especially interesting for graphs of different sizes?

### Problem 21. Extrapolation and graph size

Suppose a GNN is trained only on small graphs and tested on much larger graphs.

(a) Why might this be difficult?

(b) Why do local neighborhood patterns matter here?

(c) Give one reason a model might fail even if it had very low training error.

### Problem 22. Algorithmic alignment

The survey points out that message passing GNNs can align naturally with iterative graph algorithms such as shortest path updates.

(a) What does it mean for a model architecture to be aligned with a task?

(b) Why might an iterative message passing architecture be a good fit for shortest path computation?

(c) Give one other graph task for which message passing seems natural.

---

## Part VI. Reflection (10 points)

Write a short paragraph (5–8 sentences) answering the following prompt:

> What did you learn from manually running both a GNN and the Weisfeiler–Leman algorithm? Did anything about their strengths or limitations surprise you?

---

## Submission guidelines

Submit one PDF containing your work for this Part of the Homework. For all hand computations, show intermediate steps. For conceptual questions, write complete sentences, not just fragments. ws of generalization and extrapolation.